

AI药物设计

第5次课：神经网络基础

从线性到非线性 →

2025/12/12



本课大纲

第一节：从线性到非线性

1. 线性模型的局限性

药学中的非线性关系

2. 激活函数的引入

Sigmoid与ReLU

3. 单个神经元工作原理

生物学类比与数学模型

第二节：网络结构设计

4. 为什么需要多层

XOR问题与复杂模式

5. 前向传播过程

信息流动与计算实现

6. 反向传播原理

梯度计算与参数更新

课程信息

基本信息

- 📖 课程名称: AI药物设计
- 📅 第5次课: 神经网络基础
- 🎓 目标学生: 药学院大三学生
- 🕒 课程时长: 90分钟 (45分钟×2节)

今天的目标

- 💡 为什么需要神经网络: 理解线性模型的局限性
- 🔁 激活函数: 引入非线性变换
- ⚡ 单个神经元: 掌握基本计算单元
- 🏗️ 网络结构: 设计多层网络
- ➡️ 前向传播: 理解信息流动过程
- 💊 药学应用: 将概念应用到药物设计

第一节：从线性到非线性

为什么需要神经网络？

回顾：线性回归的成功

线性回归在很多问题上表现良好：

房价预测

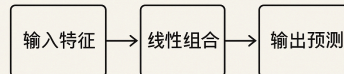
$$\text{房价} = 1.2 \times \text{面积} + 10$$

✓ 预测准确，关系简单明了

剂量-反应（低剂量）

$$\text{反应} = 0.8 \times \text{剂量} + 0.5$$

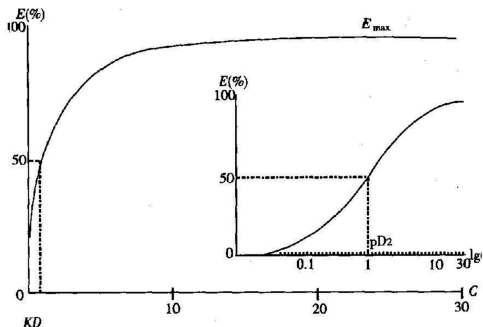
✓ 在一定范围内呈线性关系



✓ 优点：简单、可解释、计算快速

然而，现实世界是非线性的！

💊 药物剂量-效应曲线



✗ 线性模型无法拟合S型曲线！

更多非线性例子

🧪 酶促反应动力学

米氏方程: $V = V_{max} \times [S] / (K_m + [S])$

🔄 药物代谢

- 非线性清除率，饱和动力学

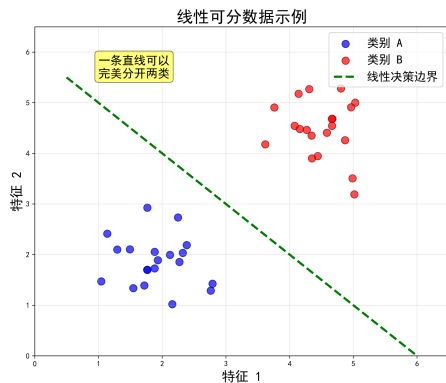
🎯 受体结合

- 协同效应，竞争性抑制

💡 需要新工具：能处理非线性关系的模型

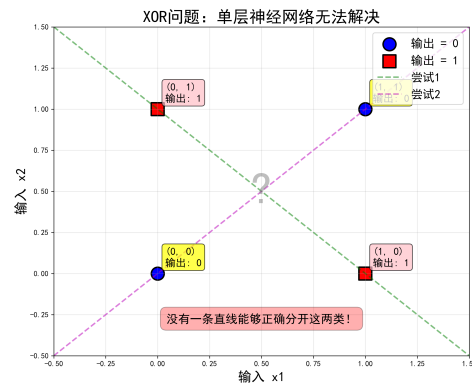
可视化：线性 vs 非线性

✓ 线性可分问题



一条直线就能分开

✗ 非线性问题 (XOR)



无法用直线分开

练习：药学中的案例

判断以下关系是线性还是非线性？

1. 药片重量 vs 有效成分含量

2. 血药浓度 vs 药效（止痛药）

3. pH值 vs 药物溶解度

关系类型分析

1. 药片重量 vs 有效成分含量

- **线性关系**，例：100mg药片含10mg有效成分，200mg含20mg

2. 血药浓度 vs 药效（止痛药）

- **非线性关系**，呈S型曲线（sigmoid），低浓度无效，达到阈值后效果显著，高浓度时效果饱和

3. pH值 vs 药物溶解度

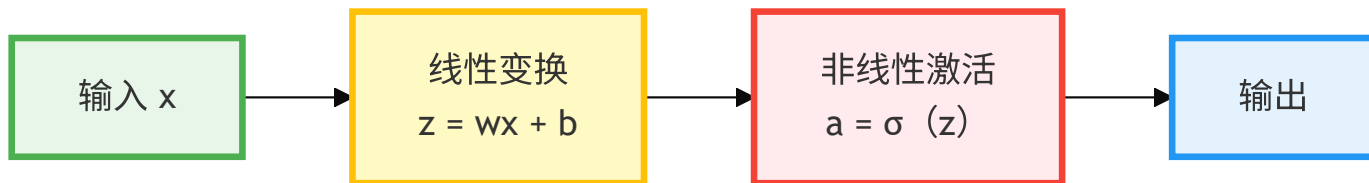
- **非线性关系**，多数药物在特定pH有最大溶解度，离开最适pH后溶解度急剧下降

激活函数的引入

让线性模型能处理非线性问题

如何引入非线性？

核心思想：在线性计算后加入非线性变换



1 第一步：线性计算

$$z = wx + b$$

与之前完全相同

2 第二步：激活函数

$$a = \sigma(z)$$

引入非线性！

Sigmoid激活函数

数学形式

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

特点

输出范围

$(0, 1)$

曲线形状

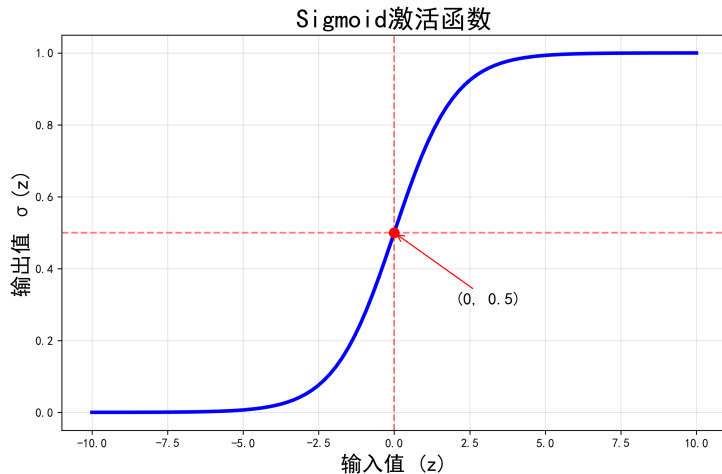
S型

导数性质

处处可导

生物学意义

明确



生物学意义：模拟了生物系统的饱和特性

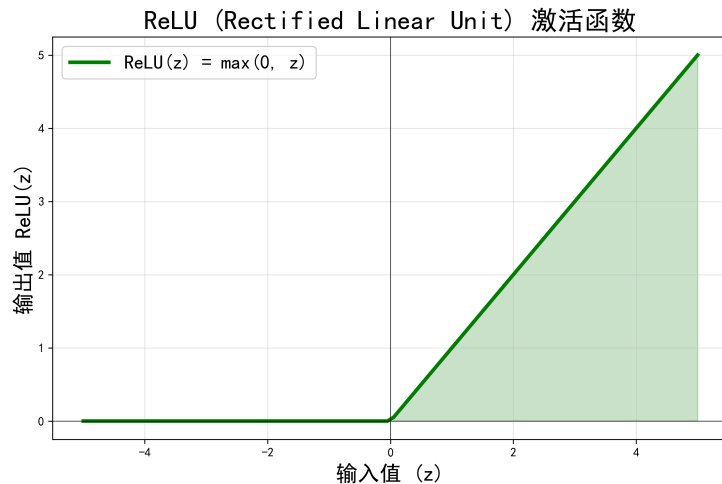
ReLU激活函数

更简单的选择

$$\text{ReLU}(z) = \max(0, z)$$

特点

- ✓ 计算简单：负数变0，正数不变
- ✓ 没有饱和区
- ✓ 梯度计算快
- ✓ 深度网络的首选

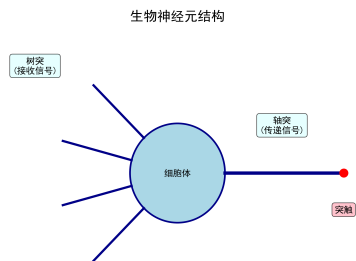


💡 优势：解决了梯度消失问题，训练更快

单个神经元工作原理

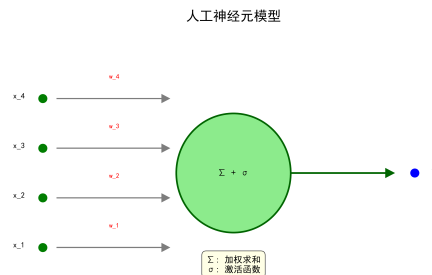
构建智能的基本单元

生物神经元



- 树突：接收信号
- 细胞体：处理信号
- 轴突：传递信号
- 突触：连接其他神经元

人工神经元



- 输入： $x_1, x_2, x_3 \dots$
- 权重： $w_1, w_2, w_3 \dots$ （突触强度）
- 求和： $\Sigma(w_i x_i) + b$
- 激活： $\sigma(z)$
- 输出： a

完整的神经元计算流程

三输入神经元示例

输入特征（药物分子）

- x_1 = 分子量 = 180
- x_2 = LogP = 2.0
- x_3 = 氢键供体数 = 3

参数

- $w_1 = -0.01, w_2 = 0.5, w_3 = -0.2$
- $b = 2.0$

计算步骤

1. 线性组合:

$$\begin{aligned} z &= w_1 \times x_1 + w_2 \times x_2 + w_3 \times x_3 + b \\ &= (-0.01) \times 180 + 0.5 \times 2.0 + (-0.2) \times 3 + 2.0 \\ &= -1.8 + 1.0 - 0.6 + 2.0 \\ &= 0.6 \end{aligned}$$

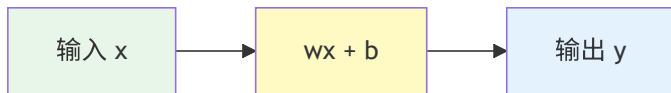
2. 激活函数:

$$a = \sigma(0.6) \approx 0.65$$

 **输出:** 该分子的预测活性为0.65（65%可能有活性）

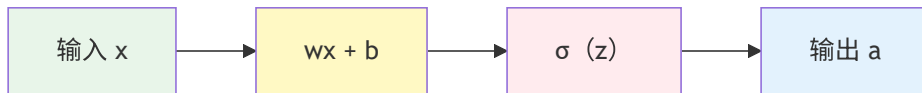
与第4课的联系

线性回归（第4课）



- 直接输出线性组合
- 只能拟合线性关系
- 参数更新：梯度下降

神经元（第5课）



- 增加激活函数
- 可以拟合非线性
- 参数更新：仍是梯度下降！

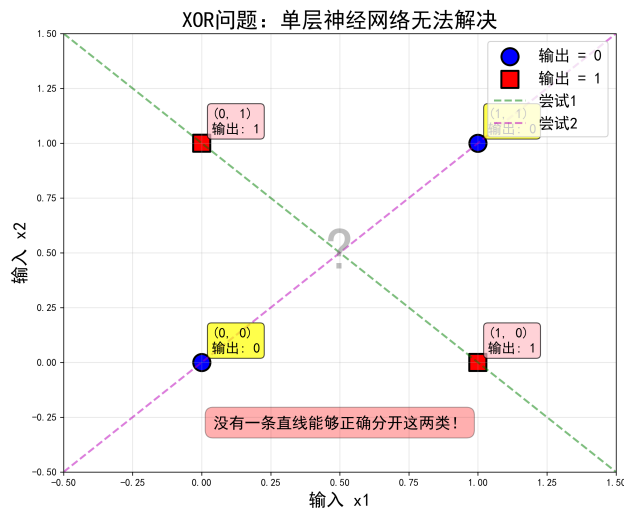
 核心联系: 神经元 = 线性回归 + 激活函数

第二节：网络结构设计

从单个神经元到神经网络

为什么需要多层?

异或 (XOR) 逻辑示意图



Tinker With a **Neural Network** Right Here in Your Browser.
Don't Worry, You Can't Break It. We Promise.

(<https://gij>)

⏮ ⏪ ⏩ ⏭

Epoch
000,000

Learning rate
0.03

Activation
Sigmoid

Regularization
None

Regularization rate
0

Problem type
Classification

DATA

Which dataset do you want to use?



FEATURES

Which properties do you want to feed in?

X_1

X_2

+ - 2 HIDDEN LAYERS

+ -
4 neurons

+ -
2 neurons

OUTPUT

Test loss 0.515
Training loss 0.507



药物发现中的复杂模式

为什么需要多层神经网络?



特征组合的必要性

单特征局限

分子量单独无法预测活性

交互作用

分子量 $\times$ 脂溶性组合更有意义

复杂模式

多特征协同决定药效

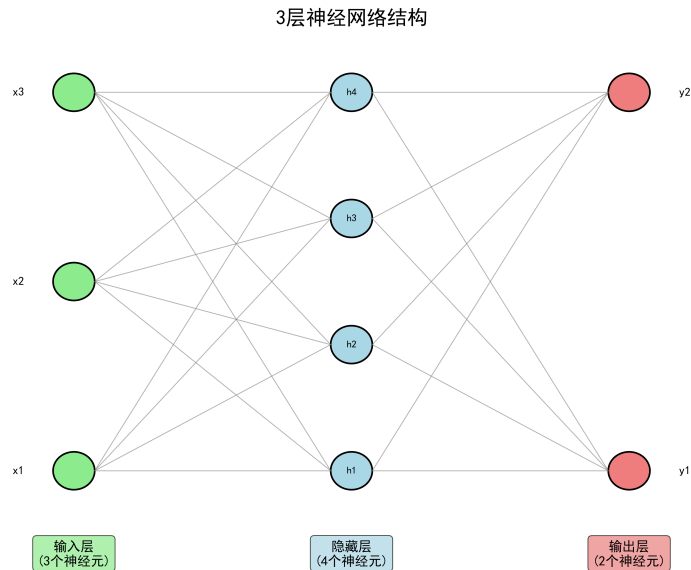
药物发现中的层次化学习

实际例子：ADMET预测



 分层抽象：从原子到分子到生物活性

三层网络架构设计



标准三层网络：输入层(3) → 隐藏层(4) → 输出层(1)

权重矩阵的维度

第一层：输入→隐藏层

权重矩阵 W_1

4×3

偏置向量 b_1

4×1

参数总数

$$4 \times 3 + 4 = 16$$

第二层：隐藏层→输出

权重矩阵 W_2

1×4

偏置 b_2

1×1

参数总数

$$1 \times 4 + 1 = 5$$

$$\text{网络总参数: } 16 + 5 = 21$$

练习：设计一个预测药物溶解度的网络

输入特征（5个）：

MW

分子量

LogP

脂溶性

HBD

氢键供体数

HBA

氢键受体数

RB

可旋转键数

请回答：

- 1 如果隐藏层用8个神经元，第一层需要多少参数？
- 2 输出层需要多少参数？
- 3 网络总共多少参数？

练习答案：药物溶解度预测网络

网络结构

5

输入层

8

隐藏层

1

输出层

1 第一层（输入→隐藏）

权重: $8 \times 5 = 40$

偏置: $8 \times 1 = 8$

小计: 48个

2 第二层（隐藏→输出）

权重: $1 \times 8 = 8$

偏置: $1 \times 1 = 1$

小计: 9个

3 总参数

57

个参数

 $48 + 9 = 57$

前向传播过程

信息在网络中的流动

多层网络的前向传播

逐层计算过程



第一层计算

参数设置

$$x = [1.0, 2.0]$$

$$W1 = \begin{bmatrix} 0.5 & -0.3 \\ 0.2 & 0.4 \end{bmatrix}$$

$$b1 = [0.1, -0.2]$$

计算过程

$$z1_1 = 0.5 \times 1.0 + (-0.3) \times 2.0 + 0.1 = 0.0$$

$$z1_2 = 0.2 \times 1.0 + 0.4 \times 2.0 + (-0.2) = 0.8$$

$$h1 = \sigma(0.0) = 0.5$$

$$h2 = \sigma(0.8) \approx 0.69$$

第二层计算

隐藏层输出

```
h = [0.5, 0.69] # 来自第一层
```

第二层参数

```
W2 = [0.6, -0.4] # 输出神经元的权重  
b2 = 0.3
```

输出计算

```
# 线性组合  
z2 = 0.6×0.5 + (-0.4)×0.69 + 0.3  
    = 0.3 - 0.276 + 0.3  
    = 0.324  
  
# 最终输出（可以用Sigmoid或直接输出）  
y = σ(0.324) ≈ 0.58
```



完整流程: 1.0, 2.0 → 隐藏层0.5, 0.69 → 输出0.58

```

import numpy as np
def sigmoid(z):
    """ $\sigma(z) = 1 / (1 + e^{-z})$ """
    return 1 / (1 + np.exp(-z))
# 🛠️ 网络参数设置
W1 = np.array([[0.5, -0.3],      # 第一层权重矩阵 (2x2)
               [0.2, 0.4]])
b1 = np.array([0.1, -0.2])      # 第一层偏置 (2x1)
W2 = np.array([0.6, -0.4])      # 第二层权重 (1x2)
b2 = 0.3                        # 第二层偏置 (1x1)
# 📥 输入数据
x = np.array([1.0, 2.0])
# 📊 第一层: 输入→隐藏
z1 = np.dot(W1, x) + b1 #  $[2, 2] \times [2] + [2] = [2]$ 
h = sigmoid(z1)         # 激活函数
print(f"🟢 隐藏层输出: {h}")
# 🎯 第二层: 隐藏→输出
z2 = np.dot(W2, h) + b2 #  $[2] \times [2] + \text{scalar} = \text{scalar}$ 
y = sigmoid(z2)         # 最终激活
print(f"🔴 网络输出: {y}")
# 📈 运行结果
# 🟢 隐藏层输出: [0.5, 0.69]
# 🔴 网络输出: 0.58

```

与线性回归对比

线性回归

```
# 单步计算  
y = np.dot(w, x) + b
```

特点

-  计算简单
-  参数少
-  只能线性
-  表达能力有限

神经网络

```
# 多步计算  
z1 = np.dot(W1, x) + b1  
h = sigmoid(z1)  
z2 = np.dot(W2, h) + b2  
y = sigmoid(z2)
```

特点

-  计算复杂
-  参数多
-  可以非线性
-  表达能力强



权衡: 复杂度换取表达能力

反向传播扩展

多层网络的梯度计算

回顾第4课的核心思想

单层的反向传播

还记得线性回归的梯度吗？

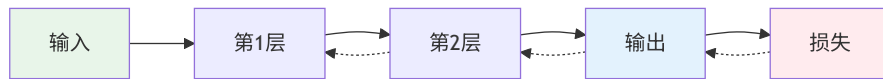
```
# 损失对参数的偏导数
 $\partial L / \partial w = 2(\hat{y} - y) \times x$ 
 $\partial L / \partial b = 2(\hat{y} - y)$ 

# 参数更新
 $w = w - \alpha \times \partial L / \partial w$ 
 $b = b - \alpha \times \partial L / \partial b$ 
```

核心思想

1. 计算损失
2. 求偏导数
3. 梯度下降更新

多层的挑战



新问题

- 如何计算第1层的梯度？
- 梯度如何向后传递？

反向传播的数学基础

神经网络的损失函数 完整的数学表达式

对于两层网络，我们有：

前向传播：

$$z_1 = W_1 x + b_1$$

$$h = \sigma(z_1)$$

$$z_2 = W_2 h + b_2$$

$$\hat{y} = \sigma(z_2)$$

损失函数：

$$L = \frac{1}{2}(\hat{y} - y)^2$$

问题：如何计算 $\frac{\partial L}{\partial W_1}$ ？

这就需要用到链式法则！

链式法则的应用

$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial z_2} \times \frac{\partial z_2}{\partial h} \times \frac{\partial h}{\partial z_1} \times \frac{\partial z_1}{\partial W_1}$$

💡 本质：逐层计算影响，然后相乘

反向传播的基本步骤

1. 前向传播

保存中间结果

```
# 第一层
z1 = W1 @ x + b1
h = sigmoid(z1) # 保存h

# 第二层
z2 = W2 @ h + b2
y = sigmoid(z2) # 保存y
```

2. 输出层梯度

从损失开始计算

```
# 损失对输出的梯度
dL_dy = 2(y - y_true)

# 输出层参数梯度
dL_dW2 = dL_dy * sigmoid'(z2) * h
dL_db2 = dL_dy * sigmoid'(z2)
```

3. 隐藏层梯度

反向传播链式计算

```
# 梯度传递到隐藏层
dL_dh = dL_dy * sigmoid'(z2) * W2

# 隐藏层参数梯度
dL_dW1 = dL_dh * sigmoid'(z1) * x
dL_db1 = dL_dh * sigmoid'(z1)
```

好消息：不用手动计算！PyTorch自动梯度

```
import torch
import torch.nn as nn
# 定义网络
class SimpleNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer1 = nn.Linear(3, 4)
        self.layer2 = nn.Linear(4, 1)
        self.sigmoid = nn.Sigmoid()
    def forward(self, x):
        h = self.sigmoid(self.layer1(x))
        y = self.sigmoid(self.layer2(h))
        return y

# 使用网络
net = SimpleNet()
x = torch.tensor([1.0, 2.0, 3.0])
y = net(x)

# 自动计算梯度！
loss = (y - target) ** 2
loss.backward() # 自动反向传播
```


核心思想总结

与第4课一致的部分

1. 损失函数: 仍然使用MSE
2. 梯度下降: 参数更新规则相同
3. 学习率: 同样重要
4. 迭代优化: 重复直到收敛

新增的复杂性

1. 多层结构: 需要逐层计算
2. 激活函数: 增加了导数计算
3. 链式法则: 梯度的传递
4. 更多参数: 管理复杂度

 **关键认识:** 神经网络训练的本质仍是梯度下降优化

课程总结

从线性到神经网络的跨越

今天学到了什么？

为什么需要神经网络

- 线性模型的局限性
- 药学中的非线性关系

激活函数

- Sigmoid: 生物学意义
- ReLU: 计算效率

单个神经元

- 线性变换 + 激活
- 与线性回归的联系

网络结构设计

- 多层的必要性
- 参数计算方法

前向传播

- 逐层计算过程
- Python实现

反向传播原理

- 链式法则
- PyTorch自动梯度

线性模型 vs 神经网络

方面	线性回归	神经网络
函数形式	$y = wx + b$	多层非线性组合
激活函数	❌ 无	✅ Sigmoid, ReLU等
表达能力	只能拟合线性	可拟合任意函数
参数数量	少 ($d+1$)	多 (取决于层数)
计算复杂度	$O(d)$	$O(\text{层数} \times \text{神经元数})$
训练方法	梯度下降	梯度下降+反向传播
应用场景	简单线性关系	复杂非线性模式

药学应用前景



分子性质预测

- 溶解度预测
- 渗透性评估
- 稳定性分析
- pKa预测



药效预测

- IC50/EC50
- 选择性指数
- 药物相互作用
- 耐药性预测



临床应用

- 个体化给药
- 疗效预测
- 不良反应
- 剂量优化

关键公式 (1/3): 激活函数

Sigmoid函数

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

输出范围: (0, 1), 呈S型曲线

常用于二分类问题的输出层

ReLU函数

$$\text{ReLU}(z) = \max(0, z)$$

简单高效, 避免梯度消失

深度神经网络的首选激活函数

关键公式 (2/3): 神经元计算

线性组合

$$z = \sum_i w_i x_i + b = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b$$

对所有输入进行加权求和，再加上偏置项

激活输出

$$a = \sigma(z)$$

通过激活函数进行非线性变换

神经元的最终输出

关键公式 (3/3): 前向传播

多层网络的计算流程

输入层 → 隐藏层:

$$h = \sigma(W_1x + b_1)$$



隐藏层 → 输出层:

$$y = \sigma(W_2h + b_2)$$

信息逐层传播，每层进行线性变换和非线性激活

参数数量计算

权重矩阵维度

$$W_l \in \mathbb{R}^{n_{l-1} \times n_l}$$

第 l 层权重矩阵：

行数 = 前一层神经元数量，列数 = 当前层神经元数量

总参数数量

$$\text{总参数} = \sum_l (n_{l-1} \times n_l + n_l)$$

思考题

❓ 问题1：激活函数选择

为什么深层网络更喜欢用ReLU而不是Sigmoid?

提示：考虑梯度消失问题...

💡 问题2：网络深度

是否网络越深越好？在药物设计中，多少层比较合适？

提示：考虑过拟合和计算成本...

</> 问题3：生物学类比

人工神经网络与生物神经网络的主要区别是什么？这些区别对药物设计有何启发？

课后作业

巩固练习

思考拓展

- 查阅资料：了解其他激活函数（tanh, LeakyReLU等）
- 思考：如何选择隐藏层神经元的数量？
- 探索：神经网络在你研究方向的应用

下次课预告

第6次课：Python神经网络药学实战

学习内容

-  从零实现神经网络
-  PyTorch基础入门
-  构建第一个深度学习模型
-  训练和评估网络
-  药物数据实战

准备工作

-  复习今天的网络结构
-  确保PyTorch已安装
-  回顾Python语法
-  回顾NumPy操作

谢谢大家！

从线性到非线性，开启深度学习之门

理解原理，掌握工具，服务药学

